GOVERNED AGENTIC AI

AUTARCH

You don't use AI. You preside over it.

The agent framework where intelligence is unlimited —
but every consequence is **governed, recorded, and provable.**

The problem nobody else is solving

AI agents are starting to take **real-world actions** — moving money, changing records, calling tools, deleting data.

Today's frameworks are built to make agents **act**.
None are built to make them **accountable.**

- 🔓 Tools run with unrestricted access
- 🌀 Agents loop, overspend, double-execute
- ❓ "What did it do, and who allowed it?" — no answer
- 🚫 Nothing a regulator or auditor can trust

> *Ungoverned agents are a **liability** waiting to happen.*

The shift

The old question	The Autarch question
"What can the AI do?"	*"What may the AI do — and can you prove it?"*

Intelligence becomes a commodity. Governance becomes the moat.

Autarch inverts the model: the AI only ever **proposes**.
A deterministic kernel **disposes.**

What is Autarch?

A **self-contained Python framework** for building AI agents whose every action is:

- ✅ **Authorized** — by an explicit capability grant
- ✅ **Bounded** — by policy and budget
- ✅ **Recorded** — in a tamper-evident ledger
- ✅ **Provable** — cryptographically, to a third party

> *Pure Python + SQLite.* ***Zero required dependencies.*** `pip install` *and it runs anywhere — fully offline.*

The Moat

Five things no other framework can do

(without re-architecting from scratch)

1 · Provable execution

Every action is **signed** into a tamper-evident ledger (Ed25519 + hash chain).

You can **prove** — to an auditor, a regulator, a court — *what* an agent did, *by whose authority*, and that the record was **not altered.**

> *No orchestration framework has a chokepoint to sign. Autarch is built around one.*

2 · Deterministic capability kernel

"AI proposes, the kernel disposes."

- Nothing acts without an explicit, ratified grant
- **Deny by default**
- Prompt injection can't escalate — the kernel gates the **action**, no matter what the prompt says

> *The model can be fooled. The kernel cannot be talked out of the rules.*

3 · Formal safety guarantees

Prove an entire class of actions is impossible — *before* running.

```
autarch guarantee --forbid file.delete
→ PROVEN: this agent can never delete a file.
```

If a guarantee can't hold, you get a **counterexample**, not a surprise in production.

> *Not testing. Not hoping. A sound static proof.*

4 · Governed evaluation

Built-in **judge-LLMs** and deterministic checks —
and the verdict is **signed into the ledger.**

Prove an output was **evaluated**, by **which judge**,
and **what it scored.**

> *Everyone else's evaluation is an ungoverned side-activity. Yours is evidence.*

5 · Right-to-be-forgotten — *with the proof intact*

Redact PII for GDPR / compliance —
while the integrity chain **still verifies.**

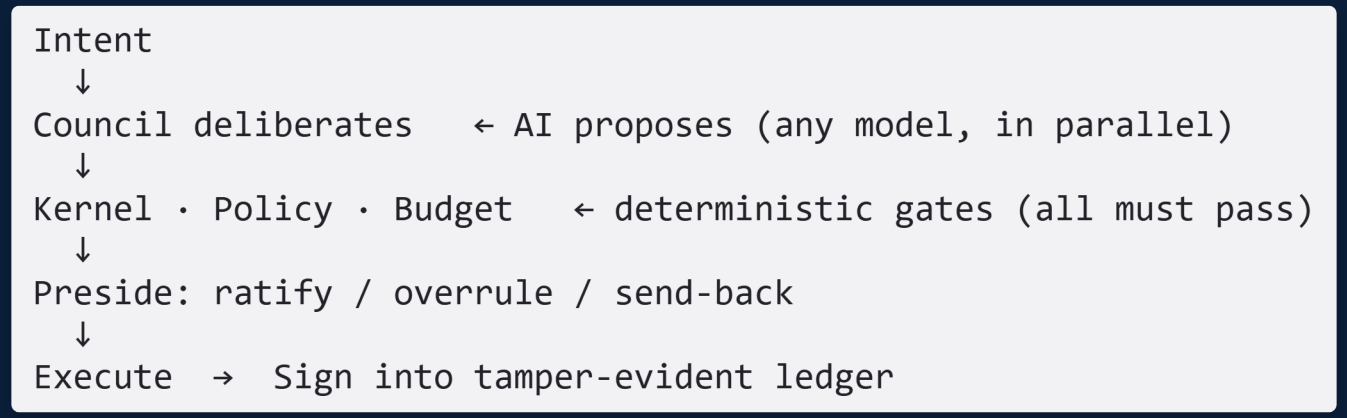
- The sensitive data is **provably gone**
- The audit proof **survives**

> *The hard part of compliance, solved: forget the data, keep the trust.*

Core governance toolkit

Capability	What it does
Capability kernel	fine-grained grants, scope, limits — deny by default
Policy-as-code	allow / deny / require-ratify rules
RBAC	roles decide *who* may wield what
Delegation	sub-agents get *strictly weaker* authority
Economic kernel	budgets every action — "*allowed ≠ affordable*"
Human-in-the-loop	ratify / overrule / send-back, with precedent

How it works



AI on top. A deterministic, provable kernel underneath.

Reliability that doesn't fall over

- 🔁 **Never crashes on rate limits** — proactive token-aware queue + retry + circuit breaker, auto-applied to every model call
- 💾 **Durable & resumable** — crash-safe; resume never double-executes
- ↩️ **Governed rewind** — audited, reversible undo
- 🧩 **Typed errors** — stable, catchable codes

> *The boring production problems that sink other agents — already handled.*

Observability & compliance

- 📡 **Structured event stream** — every step, machine-readable
- 📊 **Telemetry** — JSON-lines or OpenTelemetry
- 📜 **Regulator-grade audit export** + retention controls
- ❤️ **Health/readiness probes** — container-ready

> *Observable, exportable, forgettable, provable — without losing the integrity proof.*

Strategy: absorb, don't reinvent

Autarch sits **underneath** the tools you already use.

- 🔌 **MCP** — govern external MCP tools; serve yours *as* a governed MCP server
- 🔗 **LangChain bridge** — run their tools **governed**; expose yours to their agents

> ***Keep your LangChain. Add Autarch underneath. Now it's auditable.***

Their ecosystem becomes **your** feature set — instantly governed.

Runs anywhere

- 🪶 **Pure Python + stdlib SQLite** — zero required dependencies
- 🔒 **Local-first** — first-class Ollama; data never leaves the machine
- 🌐 **Encrypted mesh** — one identity, AES-256-GCM sync, no broker
- 🐋 **Docker + CI included**
- ⌨️ **CLI** — `do · why · prove · guarantee · health · audit · mesh`

> *From an air-gapped laptop to a container fleet — same package.*

Where it stands today

VERIFIED, NOT VAPOR

- ✅ **339 automated tests passing**
- ✅ **19 runnable examples**
- ✅ **v0.8.0** — coherent, end-to-end
- ✅ Works **100% offline**, zero runtime dependencies

> *A serious, working framework — proven on every claim above.*

How we compare

	Autarch	LangChain / AutoGen / CrewAI
Provable, signed actions	🟢 **core**	🔴 none
Deterministic kernel / deny-by-default	🟢 **core**	🔴 none
Formal safety guarantees	🟢 **unique**	🔴 none
Governed, signed evaluation	🟢 **unique**	🔴 none
GDPR redaction w/ proof intact	🟢 **unique**	🔴 none
Huge integration ecosystem	🟡 *absorbs theirs*	🟢 mature
Adoption / battle-testing	🔴 early	🟢 large

> *We don't out-feature them. We **govern** them.*

Who needs this

The buyers for whom "*prove every AI action*" is a **requirement**, not a nice-to-have:

- 🏦 **Financial services** — auditable, budgeted AI actions
- 🏥 **Healthcare** — provenance + right-to-be-forgotten
- 🛡️ **Defense & critical infrastructure** — air-gapped, deny-by-default
- 🏛️ **Regulated enterprise** — compliance you can demonstrate

> *Generic users want capability.* ***Regulated buyers will pay for control.***

Honest status & roadmap

Shipped (v0.8.0): governance, provenance, guarantees, economy, evaluation, resilience, durability, MCP/LangChain bridges, mesh.

Next (planned): provable **memory**, async runtime, structured output, multi-agent handoffs.

> *Early access / prototype — **no production users or third-party security audit yet.** We sell trust; we won't overclaim it.*

The one-liner that wins

"The only agent framework that can prove what it did."

For AI you can put in front of a **regulator.**

Autarch — govern the consequences, not the intelligence.

Thank you

Autarch · Governed, provable agentic AI

You don't use AI. You preside over it.

```
pip install autarch
```